

Crypto Market Making System Architecture Document

Quant Development Team

October 16, 2025

Abstract

This document outlines the architecture for a high-performance cryptocurrency market making system. The system is designed to operate across multiple exchanges, providing normalized market data, order management, and position tracking through a distributed, message-driven architecture using ZeroMQ and Protocol Buffers.

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions	3
2	System Overview	3
2.1	Architecture Principles	3
2.2	High-Level Architecture	4
3	Technical Requirements	4
3.1	Functional Requirements	4
3.1.1	Market Data Requirements	4
3.1.2	Order Management Requirements	4
3.1.3	Position Tracking Requirements	4
3.1.4	Strategy Requirements	5
3.2	Non-Functional Requirements	5
3.2.1	Performance	5
3.2.2	Reliability	5
3.2.3	Scalability	5
4	System Architecture	5
4.1	Component Diagram	5
4.2	Data Flow	6
4.2.1	Market Data Flow	6
4.2.2	Order Flow	6
4.2.3	Position Flow	7

5	Component Specifications	7
5.1	Market Data Service (MDS)	7
5.1.1	Responsibilities	7
5.1.2	Interfaces	7
5.2	Order Management System (OMS)	7
5.2.1	Responsibilities	7
5.2.2	Interfaces	7
5.3	Position Server	8
5.3.1	Responsibilities	8
5.3.2	Interfaces	8
5.4	Exchange Handlers	8
5.4.1	Responsibilities	8
5.4.2	Supported Exchanges	8
5.5	Feed Handlers	9
5.5.1	Base Feed Handler	9
5.5.2	Specialized Feed Handlers	9
5.6	Strategy Process	9
5.6.1	Architecture	9
6	Data Formats and Protocols	9
6.1	Protocol Buffer Schemas	9
6.1.1	Market Data Messages	9
6.1.2	Order Messages	9
6.1.3	Position Messages	10
6.2	ZeroMQ Patterns	10
6.2.1	Publisher-Subscriber	10
6.2.2	Request-Response	10
7	Configuration Management	10
7.1	Strategy Configuration	10
7.2	System Configuration	11
8	Operational Considerations	11
8.1	Deployment Architecture	11
8.2	Monitoring and Logging	11
8.3	Error Handling	11
9	Development Phases	12
9.1	Phase 1: Core Infrastructure	12
9.2	Phase 2: Trading Capabilities	12
9.3	Phase 3: Production Ready	12
10	Risk Management	12
10.1	Technical Risks	12
10.2	Mitigation Strategies	12

11 Appendices	13
11.1 Message Schema Reference	13
11.2 Exchange API Specifications	13
11.3 Deployment Guide	13

1 Introduction

1.1 Purpose

This document provides the architectural blueprint for a multi-exchange cryptocurrency market making system. It defines the component structure, communication protocols, data formats, and integration patterns to ensure consistent implementation across development teams.

1.2 Scope

The system will support:

- Real-time market data consumption from multiple crypto exchanges
- Unified order management across exchanges
- Real-time position tracking and risk management
- Multiple concurrent trading strategies
- Configurable data feed subscriptions

1.3 Definitions

OMS Order Management System - routes and manages orders across exchanges

MDS Market Data Service - collects and normalizes market data

Snapshot Book Complete order book representation sent in each update

Feed Handler Strategy-specific component for receiving market data

2 System Overview

2.1 Architecture Principles

- **Modularity:** Components are loosely coupled and independently deployable
- **Performance:** Low-latency messaging with ZeroMQ
- **Reliability:** Fault-tolerant with graceful degradation
- **Extensibility:** Easy to add new exchanges and strategies
- **Maintainability:** Clear separation of concerns and standardized interfaces

2.2 High-Level Architecture

The system follows a distributed microservices architecture with the following core components:

- **Market Data Service (MDS):** Normalizes exchange-specific data formats
- **Order Management System (OMS):** Handles order routing and execution
- **Position Server:** Tracks real-time positions across all exchanges
- **Exchange Handlers:** Exchange-specific adapters for connectivity
- **Strategy Processes:** Individual trading strategy implementations
- **Feed Handlers:** Configurable data subscriptions for strategies

3 Technical Requirements

3.1 Functional Requirements

3.1.1 Market Data Requirements

- Support real-time order book data from at least 3 exchanges (Binance, Coinbase, Kraken)
- Normalize different exchange data formats into unified protocol buffer format
- Provide snapshot order books only (no incremental updates required)
- Support trade data feeds from all connected exchanges
- Handle data quality issues and exchange disconnections gracefully

3.1.2 Order Management Requirements

- Route orders to specified exchanges with proper authentication
- Handle order acknowledgements, fills, and cancellations
- Provide real-time order status updates to strategies
- Support market and limit order types
- Implement order cancellation capabilities

3.1.3 Position Tracking Requirements

- Maintain real-time position balances across all exchanges
- Provide position query interface for strategies
- Track realized and unrealized P&L
- Support position update notifications

3.1.4 Strategy Requirements

- Support multiple concurrent strategy instances
- Allow configurable data feed subscriptions per strategy
- Provide timer-based event capabilities
- Enable request-response queries for positions and order status

3.2 Non-Functional Requirements

3.2.1 Performance

- Market data processing latency: ≤ 10 ms end-to-end
- Order response latency: ≤ 100 ms round trip
- System availability: 99.9% during trading hours
- Support for 100,000+ messages per minute

3.2.2 Reliability

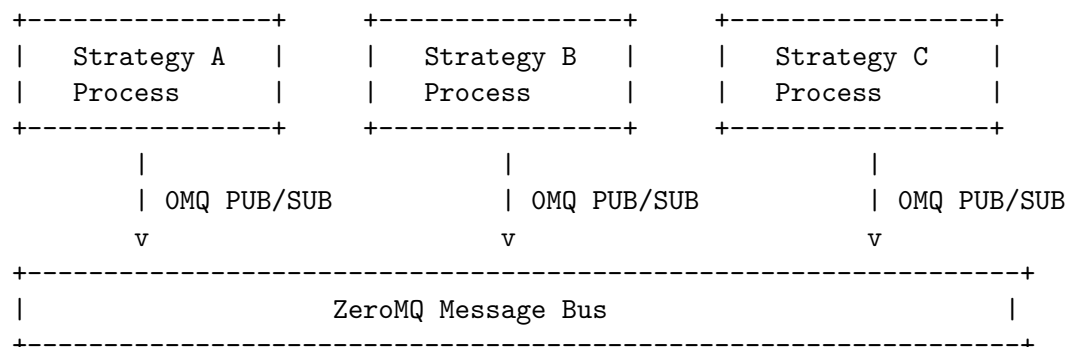
- Automatic reconnection to exchange WebSocket feeds
- Graceful handling of exchange rate limits
- Data gap detection and recovery procedures
- Circuit breaker patterns for volatile market conditions

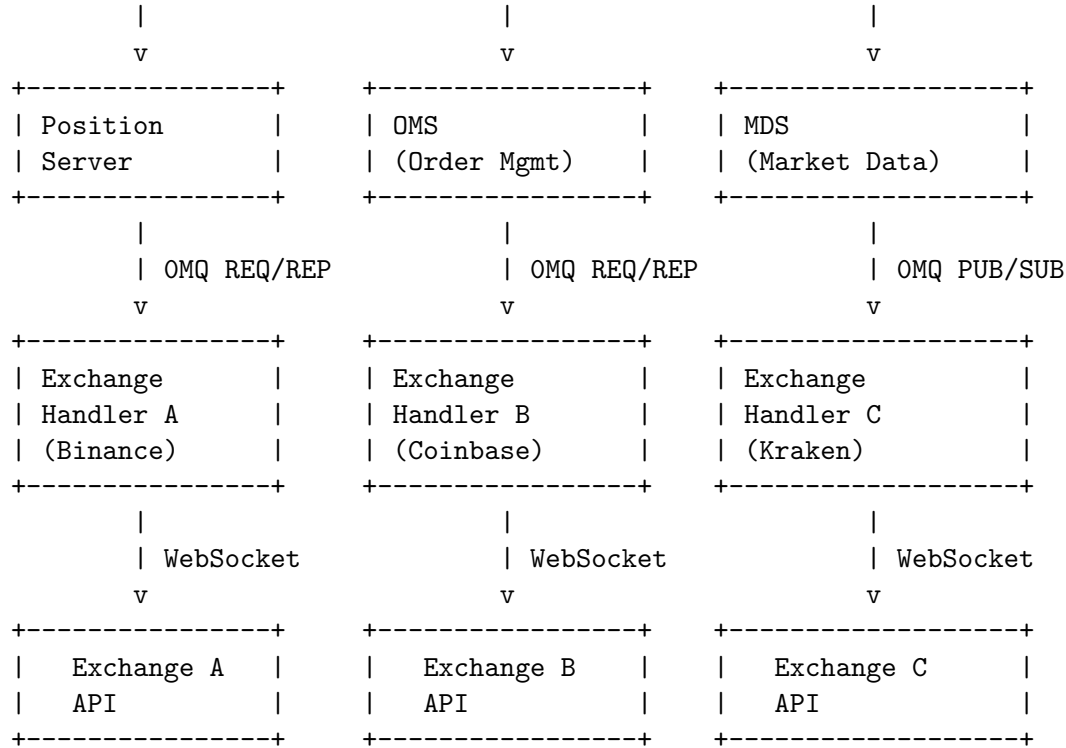
3.2.3 Scalability

- Support for 10+ concurrent strategies
- Ability to add new exchanges with minimal code changes
- Horizontal scaling of strategy processes

4 System Architecture

4.1 Component Diagram





4.2 Data Flow

4.2.1 Market Data Flow

1. Exchange WebSocket feeds push JSON market data
2. Exchange Handlers normalize JSON to Protocol Buffers
3. MDS publishes normalized data to ZeroMQ topics
4. Feed Handlers subscribe to relevant topics based on strategy configuration
5. Strategies receive callbacks (onBook, onTrade) with normalized data

4.2.2 Order Flow

1. Strategy creates OrderRequest message
2. OMS receives order via ZeroMQ request
3. OMS routes to appropriate Exchange Handler
4. Exchange Handler sends order via exchange WebSocket/REST API
5. Exchange sends order response via WebSocket
6. Exchange Handler normalizes response to Protocol Buffer
7. OMS publishes OrderResponse via ZeroMQ
8. Strategy receives order status via callback

4.2.3 Position Flow

1. Exchange Handlers receive position updates via WebSocket
2. Position Server aggregates updates across exchanges
3. Strategies can query positions via request-response
4. Position Server publishes position updates via ZeroMQ

5 Component Specifications

5.1 Market Data Service (MDS)

5.1.1 Responsibilities

- Manage WebSocket connections to exchange market data feeds
- Normalize exchange-specific JSON formats to unified Protocol Buffers
- Detect and handle data quality issues (gaps, staleness)
- Publish normalized data to appropriate ZeroMQ topics

5.1.2 Interfaces

- **Input:** Exchange WebSocket feeds (JSON)
- **Output:** ZeroMQ PUB socket with Protocol Buffer messages
- **Topics:** `orderbook.[exchange].[symbol]`, `trade.[exchange].[symbol]`

5.2 Order Management System (OMS)

5.2.1 Responsibilities

- Receive order requests from strategies
- Route orders to appropriate exchange handlers
- Manage order state machine (pending, filled, cancelled, rejected)
- Broadcast order responses to strategies
- Handle order cancellations and amendments

5.2.2 Interfaces

- **Input:** ZeroMQ REQ socket for order requests
- **Output:** ZeroMQ PUB socket for order responses
- **Internal:** Connection to exchange handlers for order routing

5.3 Position Server

5.3.1 Responsibilities

- Aggregate position updates from all exchanges
- Maintain real-time position database
- Respond to position queries from strategies
- Publish position updates to subscribers
- Calculate P&L metrics

5.3.2 Interfaces

- **Input:** ZeroMQ SUB socket for position updates
- **Output:** ZeroMQ PUB socket for position broadcasts
- **Query:** ZeroMQ REP socket for position queries

5.4 Exchange Handlers

5.4.1 Responsibilities

- Establish and maintain WebSocket connections to exchanges
- Convert exchange-specific protocols to unified format
- Handle authentication and rate limiting
- Manage reconnection logic and error handling
- Normalize market data and order responses

5.4.2 Supported Exchanges

- Binance: WebSocket for market data, REST API for orders
- GRVT: WebSocket for market data, REST API for orders
- OKex: WebSocket for market data, REST API for orders
- Hyperliquid: WebSocket for market data and orders
- Deribit: WebSocket for market data, REST API for orders

5.5 Feed Handlers

5.5.1 Base Feed Handler

- Manage ZeroMQ subscriptions based on configuration
- Deserialize Protocol Buffer messages
- Route messages to appropriate callbacks
- Provide timer event registration
- Handle request-response patterns for queries

5.5.2 Specialized Feed Handlers

- **BookFeedHandler**: Order book data only
- **TradeFeedHandler**: Trade data only
- **StrategyFeedHandler**: Combined feeds for full strategy needs

5.6 Strategy Process

5.6.1 Architecture

- Each strategy runs as independent process
- Configurable feed subscriptions via JSON configuration
- Event-driven programming model with callbacks
- Timer-based events for periodic actions
- Direct order submission interface

6 Data Formats and Protocols

6.1 Protocol Buffer Schemas

6.1.1 Market Data Messages

- **OrderBookSnapshot**: Complete order book with bids/asks
- **Trade**: Individual trade execution
- **Timestamp**: Nanosecond precision timing
- **PriceLevel**: Individual price level in order book

6.1.2 Order Messages

- **OrderRequest**: New order creation
- **OrderResponse**: Order status updates
- **OrderCancel**: Order cancellation request

6.1.3 Position Messages

- **PositionUpdate**: Real-time position changes
- **PositionQuery**: Position information request
- **PositionResponse**: Query response with positions

6.2 ZeroMQ Patterns

6.2.1 Publisher-Subscriber

- **Usage**: Market data broadcasts, order responses, position updates
- **Pattern**: One-to-many broadcast
- **Topics**: Structured as [message-type].[exchange].[symbol]

6.2.2 Request-Response

- **Usage**: Order submission, position queries, system commands
- **Pattern**: Synchronous request with response
- **Timeout**: Configurable response timeouts

7 Configuration Management

7.1 Strategy Configuration

```
{
  "strategy_id": "mm_btc_usd",
  "feeds": {
    "orderbook": [
      "tcp://localhost:5555",
      "tcp://localhost:5556"
    ],
    "trade": [
      "tcp://localhost:5557"
    ],
    "order_response": "tcp://localhost:5558",
    "position": "tcp://localhost:5559"
  },
  "parameters": {
    "spread_bps": 2.0,
    "quote_interval_ms": 1000,
    "max_position": 10.0
  },
  "servers": {
    "oms": "tcp://localhost:5560",
    "position_server": "tcp://localhost:5561"
  }
}
```

```
}  
}
```

7.2 System Configuration

- Exchange API credentials and rate limits
- ZeroMQ endpoint definitions
- Logging levels and destinations
- Performance monitoring settings

8 Operational Considerations

8.1 Deployment Architecture

- Each core component (OMS, MDS, Position Server) as separate process
- Exchange handlers as child processes or threads
- Strategy processes independently deployable
- All components deployable on single server or distributed

8.2 Monitoring and Logging

- Latency metrics for all data flows
- Order fill rates and rejection tracking
- Exchange connectivity status
- Strategy performance metrics
- Comprehensive audit logging for compliance

8.3 Error Handling

- WebSocket reconnection with exponential backoff
- Order timeout and cancellation policies
- Data gap detection and recovery procedures
- Circuit breakers for volatile market conditions

9 Development Phases

9.1 Phase 1: Core Infrastructure

- Protocol Buffer schema definition
- ZeroMQ communication framework
- Base Feed Handler implementation
- Single exchange integration (Binance)
- Basic position server

9.2 Phase 2: Trading Capabilities

- OMS implementation
- Additional exchange integrations
- Advanced Feed Handlers
- Basic market making strategy

9.3 Phase 3: Production Ready

- Risk management system
- Performance optimization
- Comprehensive monitoring
- Multiple strategy support

10 Risk Management

10.1 Technical Risks

- Exchange API changes and downtime
- Network latency and connectivity issues
- Data quality and feed reliability
- System resource constraints

10.2 Mitigation Strategies

- Multiple exchange redundancy
- Comprehensive error handling and reconnection logic
- Data validation and quality checks
- Resource monitoring and alerting

11 Appendices

11.1 Message Schema Reference

Complete Protocol Buffer message definitions and field descriptions.

11.2 Exchange API Specifications

Detailed exchange-specific integration requirements and rate limits.

11.3 Deployment Guide

Step-by-step deployment instructions for development and production environments.